


```
from matplotlib.offsetbox import OffsetImage, AnnotationBbox
import cv2
img2vec = Img2Vec() # инициализируем вектора, загрузится предобученная сеть.
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_t
102967424/102967424 — 0s 0us/step

```
image_paths = []
image_classes = ['pizza', 'schooner', 'scissors', 'panda', 'brain'] # выберем классы для отображения, провер
# считываем пути к изображениям этих классов
for image_class in image_classes:
    image_paths.extend(glob.glob('101_ObjectCategories/' + image_class + '/*.jpg')) # только .jpg
# сколько их
len(image_paths)
```

291

```
image_vectors = {}
for image_path in image_paths: # для каждого файла
    vector = img2vec.get_vec(image_path) # получаем векторное представление
    image_vectors[image_path] = vector # добавляем в словарь, название=ключ.
```

```
1/1 ————— 0s 44ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 47ms/step
1/1 ————— 0s 47ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 53ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 71ms/step
1/1 ————— 0s 67ms/step
1/1 ————— 0s 75ms/step
1/1 ————— 0s 62ms/step
1/1 ————— 0s 63ms/step
1/1 ————— 0s 70ms/step
1/1 ————— 0s 60ms/step
1/1 ————— 0s 63ms/step
1/1 ————— 0s 58ms/step
1/1 ————— 0s 62ms/step
1/1 ————— 0s 65ms/step
1/1 ————— 0s 61ms/step
1/1 ————— 0s 115ms/step
1/1 ————— 0s 167ms/step
1/1 ————— 0s 71ms/step
1/1 ————— 0s 66ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 98ms/step
1/1 ————— 0s 89ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 53ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 42ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 52ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 44ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 43ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 45ms/step
1/1 ————— 0s 48ms/step
```

```
X = np.stack(list(image_vectors.values()))
pca_50 = PCA(n_components=50) # первые 50 главных компонент
pca_result_50 = pca_50.fit_transform(X) # и проецируем на них
print('Cumulative explained variation for 50 principal components: {}'.format(np.sum(pca_50.explained_variance_ratio_)))
```

```
print(np.shape(pca_result_50))
# 2 компоненты TSNE
tsne = TSNE(n_components=2, verbose=1, n_iter=3000)
tsne_result = tsne.fit_transform(pca_result_50) # проецируем
```

```
Cumulative explained variation for 50 principal components: 0.7659825682640076
(291, 50)
```

```
[t-SNE] Computing 91 nearest neighbors...
```

```
[t-SNE] Indexed 291 samples in 0.002s...
```

```
[t-SNE] Computed neighbors for 291 samples in 0.103s...
```

```
[t-SNE] Computed conditional probabilities for sample 291 / 291
```

```
[t-SNE] Mean sigma: 12.823326
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to 'max_iter' in 1.0.0. Please update your code. Use warnings.warn()
```

```
[t-SNE] KL divergence after 250 iterations with early exaggeration: 51.029602
```

```
[t-SNE] KL divergence after 1450 iterations: 0.342569
```

```
tsne_result_scaled = StandardScaler().fit_transform(tsne_result) # масштабируем
images = []
```

```
for image_path in image_paths:
```

```
    image = cv2.imread(image_path, 3) # загружаем изображение
```

```
    b,g,r = cv2.split(image) # разделяем каналы b, g,
```

```
    image = cv2.merge([r,g,b]) # собираем в r, g, b
```

```
    image = cv2.resize(image, (50,50)) # масштаб
```

```
    images.append(image) # добавляем в список
```

```
# рисуем
```

```
fig, ax = plt.subplots(figsize=(20,15))
```

```
artists = []
```

```
for xy, i in zip(tsne_result_scaled, images): # перебираем TSNE вектора (это двумерные координаты) и изображ
```

```
    x0, y0 = xy # разделяем горизонтальную и вертикальную координату
```

```
    img = OffsetImage(i, zoom=.7) # задаем графический объект с картинкой
```

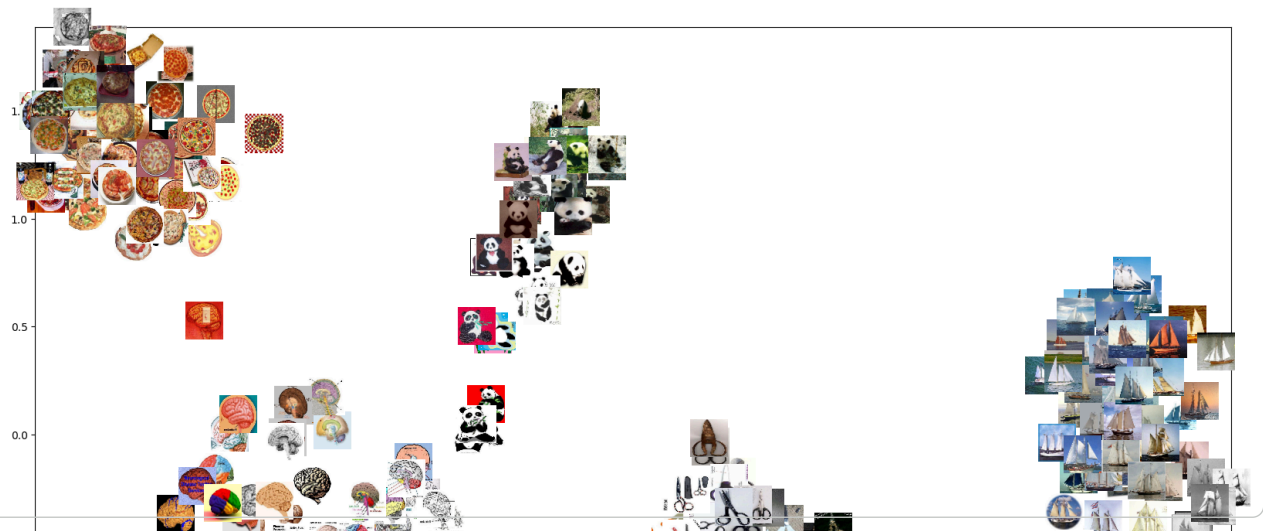
```
    ab = AnnotationBbox(img, (x0, y0), xycoords='data', frameon=False) # вставляем эту картинку по координатам
```

```
    artists.append(ax.add_artist(ab)) # добавляем на график
```

```
ax.update_datalim(tsne_result_scaled) # пределы отображения
```

```
ax.autoscale(enable=True, axis='both', tight=True) # автомасштаб
```

```
plt.show() # отображаем
```



```
# путь к первому изображению
image_path1='101_ObjectCategories/Leopards/image_0001.jpg'
# его вектор, должен быть размером 1на2048
vector1 = np.reshape(img2vec.get_vec(image_path1),(1,-1))
# путь ко второму изображению
image_path2='101_ObjectCategories/Motorbikes/image_0001.jpg'
# его вектор, должен быть размером 1на2048
vector2 = np.reshape(img2vec.get_vec(image_path2),(1,-1))
# их похожесть, чем больше тем похожей
similarity_matrix = cosine_similarity(vector1, vector2)
print('для разных ',similarity_matrix[0][0])
# путь к третьему изображению
image_path3='101_ObjectCategories/Motorbikes/image_0005.jpg'
# его вектор, должен быть размером 1на2048
vector3 = np.reshape(img2vec.get_vec(image_path3),(1,-1))
# их похожесть, чем больше тем похожей
similarity_matrix = cosine_similarity(vector3, vector2)
print('для похожих ',similarity_matrix[0][0])
```

```
1/1 ————— 0s 67ms/step
1/1 ————— 0s 79ms/step
для разных  0.25689077
1/1 ————— 0s 75ms/step
для похожих  0.8999974
```